

Random Forest Based Intrusion Detection System

Bojun Song

Shaanxi University of Science and Technology, Xian, Shaanxi, 710021, China
OrangeCrabs@outlook.com

Abstract—This article chooses to use the random forest algorithm to improve the performance of network intrusion detection systems (IDS). The algorithm significantly improves the accuracy, recall and precision of network intrusion detection compared to traditional methods. The required data and experimental results were obtained from the LUFlow dataset by using a more accurate feature extraction method. Eventually, the readability and comprehension of the experimental results were enhanced by visualizing them. Overall, the performance of the network IDS based on the random forest method has been significantly improved. However, there are still some problems in the experiment, such as the lack of comparison with other commonly used intrusion detection methods or algorithms. Similar problems make the experiment lack of comprehensiveness. Therefore, future research should consider introducing more kinds of intrusion detection methods for comparative analysis to further validate and improve the performance of the system. In addition, extending the dataset of the experiments and improving the feature extraction techniques may also bring additional improvements. In summary, although the performance of the random forest-based network IDS has been improved, there is still much room for improvement and research potential.

Index Terms—Intrusion detection system random forest feature selection categorization.

I. INTRODUCTION

Cyberattacks may not be so common in our daily lives that we think they are out of reach. However, the truth is that cyberattacks happen frequently every day. Detecting network intrusions is necessary to better protect our privacy when using the Internet. Intrusion detection technology can improve network security protection by detecting network traffic in real time and discovering potential attacks. At present, traditional intrusion detection methods have certain limitations in detection accuracy and efficiency. Therefore, researchers continue to explore new algorithms and techniques to improve the performance of intrusion detection systems (IDS). Random forest algorithm, as an integrated learning method, significantly improves the accuracy of detection by voting the classification results through multiple decision trees [1]. The process of feature selection is to filter out the most relevant features from a large amount of network data and explain the impact of these features on intrusion detection decisions. Feature selection, reduces the computational cost of feature construction, thus improving the efficiency and interpretability of the system [2].

This paper will explore how the advantages of the random forest algorithm can be utilized in conjunction with accurate feature selection techniques to improve the detection accuracy, recall and precision of network IDS. The significance of the research is that by improving the performance of network

intrusion detection, users' privacy and data security can be more effectively protected against increasingly sophisticated network attacks. In addition, this study also aims to provide new ideas and methods for future network security research and to promote the further development of the field.

II. RANDOM FOREST ALGORITHM AND SELECTION OF FEATURES

A. Introduction to the Random Forest Algorithm

Random forest is an algorithm that integrates multiple decision trees through the idea of integrated learning. Random forest algorithm is a decision tree ensemble method [3]. It learns multiple independent decision trees based on the decision trees, and then votes or averages the results of the decision trees to get the final prediction.

The random forest algorithm is an integrated learning method. The core idea of this algorithm is to build multiple independent decision trees, which are usually called as "weak learners". Each decision tree is an independent classification or regression tree that solves its own problem. In random forest, we use random sampling and feature selection to build these trees and ensure that they are independent of each other. This makes each tree learn different aspects and features of the data, each number can have its own observations, learning from different perspectives leads to tree diversity. Also, random feature selection is a key strategy that ensures that each tree is split considering only a random subset of features. This allows different trees to make decisions at different nodes using different features, improving the tree-to-tree variability. Also by randomly selecting features, each tree receives the restriction of a subset of features, thus improving the generalization of the model. Once all the decision trees have been built, Random forest polls or averages their predictions to arrive at a final prediction. This integration process allows us to obtain a "strong learner" from multiple "weak learners" because different trees may make different mistakes, but with the integrated voting used in this paper, these mistakes can be validated against each other and canceled out, improving the model's performance and robustness. On a single computer, a multicore CPU can be used to build multiple trees at the same time, which can be built independently in parallel without affecting each other. This parallel learning helps to increase the training speed [4].

There are multiple reasons for choosing the random forest algorithm. On the one hand, by integrating multiple decision trees, Random forest effectively reduces the overfitting problem that may arise from a single decision tree, thus

significantly improving the accuracy and robustness of the model. As shown in the experimental results, Random forest performs well in handling both normal and malicious traffic with a low error rate. On the other hand, Random forest is capable of handling high-dimensional data, which is particularly important in network intrusion detection. Through a random feature selection strategy, Random forest can adapt to high-dimensional data and improve the generalization ability of the model. In addition, Random forests can evaluate the importance of individual features and help identify which features are most critical for model prediction, which is valuable for optimizing feature selection and improving model performance. Random forests also have strong noise immunity, and can maintain high accuracy even when the data contains noise. Finally, the random forest algorithm supports parallel computing, which can make full use of the computational power of multi-core CPUs to accelerate the training speed, which is a significant advantage for network IDS dealing with large-scale datasets.

B. Feature Selection

Combining the filtering method and the internal method to select the feature importance index based on the internal calculation of random forest can effectively select the important features affecting the target variables, remove the impurity features in the data, and improve the performance of the model [5].

1) *Filtration method*: The Pearson correlation coefficient, a measure of correlation between features and labels, is first counted in the following steps: (1) Import the dataset. (2) Calculate for numerical or categorical features. For numerical features: directly calculate the Pearson correlation coefficient between eigenvalues and label values. For categorical features: first convert the categories to dummy variables using unique heat coding, then calculate the correlation coefficients. (3) Use the correlation function in numpy to calculate to get the correlation coefficient matrix between features. (4) Record the correlation coefficients into a list or coefficient matrix as the basis for feature selection. After counting the data, each feature and label is ranked according to its correlation metric value from high to low. A threshold is set and features with correlation greater than the threshold are selected. It can perform fast coarse-grained selection, reduce irrelevant low-quality features, and provide support for subsequent algorithms for dimensionality reduction [6].

2) *Internal method*: (1) The model of the random forest is first constructed, at which point no predetermined feature selection is performed. (2) Next by recording the importance score for each feature in the decision tree. Record the degree of decrease in prediction accuracy and root mean square error when feature values change. (3) The results of the different importance assessments for each decision tree were averaged as the final feature importance score. (4) Rank the feature importance from highest to lowest. A threshold is set and features with correlations greater than the threshold are selected.

Feature selection is done by these methods [7]. Retrain the random forest model on the selected features and test to evaluate whether the performance is improved or not.

3) *Hybrid Feature Selection*: Hybrid feature selection utilizes the advantages of both filtering and internal methods to improve the performance of feature selection. For this approach, the specific measures that can be taken are (1) Train a model using the internal method before using the filtering method. After the model has been filtered once by the feature importance score, the filtered feature set will be further selected by the filtering method. (2) Alternate between the internal method and the filtering method, and perform the feature selection in several iterations to improve the performance of the feature selection. (3) Integrate the results of the internal method and the filtering method in order to obtain comprehensive feature selection results. For example, we can use the feature importance score of the model and the feature evaluation index of the filtering method to assign weights to each feature, and finally select features according to these weights.

III. INTRUSION DETECTION MODELING

IDS is defined as a malicious, externally induced operational fault [8], IDS is a computer security technology used to monitor and identify malicious activities, attacks or anomalies in a computer network or system. It aims to detect and report potential intrusions through real-time monitoring and analysis of network traffic, system logs and other relevant data. IDS are usually based on two main detection methods: signature detection and anomaly detection [9]. IDS can be deployed at the network boundary, at the host side or inside the network to provide comprehensive security monitoring.

The following is the modeling process of the IDS: [10] (1) Data preprocessing, splitting the dataset into training set and test set. (2) Perform feature selection, using filtering to initially select relevant features, and then using a random forestbased algorithm to internally calculate the importance of features for further feature selection. (3) Train the random forest model, specify the number of trees and set other hyperparameters such as maximum depth and minimum number of samples. Finally, the model is built using the training data. (4) Perform validation and tuning of the model, testing for accuracy and recall on the validation set. And verify whether the performance of the model can be improved by tuning the hyperparameters. (5) Interpret the model results.

IV. DESCRIPTION OF DATASETS AND INTERPRETATION OF INDICATORS

By correlating with the Threat Intelligence Service, LUFlow is a flow-based intrusion detection dataset that contains emerging attack vectors against Lancaster University's address space that are composed of telemetry data collected through honeypots. Autonomous tagging mechanisms allow telemetry data to be continuously captured and tagged, and then published to this repository. Traffic that could not be recognized as malicious but did not match normal telemetry characteristics was flagged as outliers. The purpose of including this data

TABLE I
SUBSCRIPTION FOR INDICATORS

Name	Subscription	Name	Subscription
src_ip	The source IP associated with the data stream.	Packet_out	Statistics on the number of packets in transit from destination to source.
src_port	The port number of the source associated with the data stream.	entropy	The entropy of a data field in a data stream, measured in bits per byte. The value ranges from 0 to 8.
dest_ip	The destination IP address associated with the data stream.	all_entropy	The total entropy of a data field in a data stream, measured in bytes.
dest_port	The port number of the destination associated with the data stream.	aver_ipt	Average of inter-packet arrival times for flows
protocol	The number used to identify the protocol used in network communication. For example UDP is 17.	time_originate	The start time of the data stream is measured in seconds from the reference starting point
byte_in	The number of bytes of data passed from the source to the destination during transmission.	time_terminate	The end time of the data stream is measured in seconds from the reference starting point
byte_out	The number of bytes of data returned from the destination to the source during transmission.	duration	Data flow duration with microsecond accuracy
Packet_in	Statistics on the number of packets in transit from source to destination	label	Data flow labels that can be categorized as benign, anomalous and harmful, as determined by Tangerine

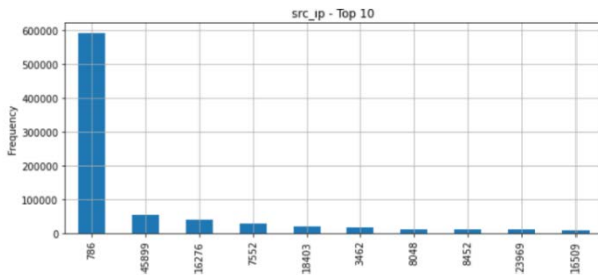


Fig. 1. The 10 most frequently occurring Source IP addresses.

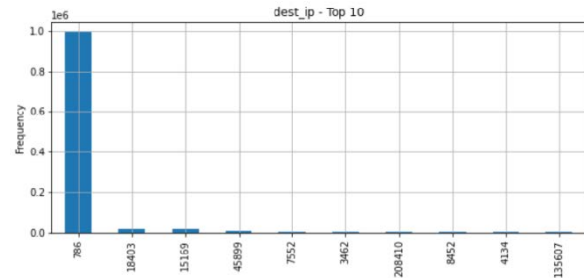


Fig. 2. The 10 most frequently occurring Destination IP addresses.

in the dataset is to encourage further analysis to reveal the true intent behind it. In addition, known normal traffic for example SSH and database traffic can be captured in the production service and included in this dataset. This dataset is constantly updated using the Citrus framework. The repository is structured by the year and month in which the telemetry was captured. For example, to find telemetry captured in September 2020, use the folder 2020/09.

V. PERFORMANCE RESULTS

A. Data Visualization Methods

There are many different methods and techniques for data visualization, depending on the type of data, the goal of the analysis, and the amount of information expected to be reached. For example, Scatterplots look at the distribution and correlation of data while also showing the relationship between two continuous variables. In this case, I will use bar charts to present categorical data to visualize the comparison of different categories. In order to display the results of the data, we first need to determine the type of data we want to show in the bar chart, in this case the port numbers. Next, we need to organize and analyze the data in a table with two

columns, one for the port number and one for the number of occurrences. We will then use Python's Matplotlib library to help us create a bar chart. Finally, add labels such as the x -axis and y -axis names and titles of the bars, and add colors and legends if needed to increase the readability of the chart. The graph is now complete and can be used to visualize the results of our experiment. In this article, we show the most common source and target IP addresses in a bar chart in Figs. 1 and 2 to deduce the most likely attack addresses.

B. Performance Results Report

Conclusions and data from the training set are derived from the training of the training set:

TABLE II
PRESENTATION OF TRAINING DATA RESULTS FOR THE TRAINING SET

	Benign	Malicious	Outlier	Error	Rate
0	409166.0	101.0	5.0	0.0003	0.0003
1	133.0	372884.0	59.0	0.0005	0.0005
2	18.0	3634.0	70444.0	0.049	0.0493
3	409317.0	376619.0	70508.0	0.005	0.0046

For cross-indicator validation summarization:

TABLE III
CROSS-INDICATOR VALIDATION DATA

	<i>Mean</i>	<i>SD</i>	<i>Cv_1</i>	<i>Cv_2</i>	<i>Cv_3</i>
accuracy	0.999	0.0001	0.999	0.999	0.999
err	0.0003	0.0001	0.0004	0.0005	0.0002
err_count	65.8	25.821	77.0	86.0	39.0
logloss	0.044	0.002	0.045	0.045	0.047
max_per_class	0.003	0.002	0.004	0.005	0.001
_error					
mean_per_class	0.999	0.0006	0.999	0.998	0.999
_accuracy					
mean_per_class	0.001	0.0006	0.001	0.002	0.0005
_error					
mse	0.006	0.0004	0.006	0.006	0.007
R^2	0.985	0.0009	0.985	0.985	0.984
RMSE	0.078	0.006	0.080	0.079	0.081

For the above experimental results we can explain: for the dataset, the training set contains samples of normal traffic (benign), malicious traffic (malicious) and abnormal traffic (outlier). Among them, there are 409,166 samples for normal traffic, 101 samples for malicious traffic and 5 samples for outlier traffic. For the error rate: the overall error rate is 0.0003, i.e. out of 409,272 samples, 106 samples are misclassified. The error rate for malicious traffic is 0.0005, i.e. out of 373,076 samples of malicious traffic, 192 were misclassified. The error rate for anomalous traffic was 0.049, i.e., out of 74,096 samples of anomalous traffic, 3,652 were misclassified

VI. CONCLUSION

Based on the results of the training and test sets, we can find that the normal traffic samples are classified very well with a very low error rate, which indicates that the classifier is able to accurately label normal traffic as normal. Meanwhile, the error rate of malicious traffic samples is also relatively low, indicating that the classifier shows some accuracy in detecting and recognizing malicious traffic. However, the high error rate of the abnormal traffic samples indicates that the classifier has some difficulty in detecting and recognizing abnormal traffic. Overall, our classifier exhibits high accuracy and low error rate in general.

Nevertheless, there is still room for improvement in detecting and recognizing anomalous traffic. Future research can focus on improving the classifier's ability to categorize

anomalous traffic, thereby improving the overall intrusion detection performance. Specifically, more advanced feature selection methods can be explored to further optimize the performance of the classifier. In addition, introducing more datasets and conducting larger-scale experiments can help us evaluate the performance of the classifier more comprehensively and find directions for further improvement.

Overall, although our classifier has shown some superiority, algorithm optimization and improvement are still needed to cope with the ever-changing cyber threats. Future work will be devoted to further improving the robustness and adaptability of the classifier in order to provide reliable intrusion detection services in more complex and changing network environments. Through continuous research and innovation, we are confident that we will be able to further improve the level of network security and provide users with a more secure network usage environment.

REFERENCES

- [1] Y. Cao, G. Zhu, X. Qi, et al. A study on random forest based classification for intrusion detection, *Computer Science*, vol. 48, no. S1, 459–463, 2001.
- [2] J. Zhang, Z. Mohammad, and H. Anwar, "Random-forests-based network intrusion detection systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* vol. 38.5, pp. 649–659, 2008.
- [3] M. Schonlau, and Y. Z. Rosie, "The random forest algorithm for statistical learning", *The Stata Journal* vol. 20, no. 1, pp. 3–29, 2020.
- [4] J. Yu, "Academic Performance Prediction Method of Online Education using Random Forest Algorithm and Artificial Intelligence Methods", *International Journal of Emerging Technologies in Learning* vol. 15, no. 5, 2021.
- [5] Hasan, Md A. M., et al. "Feature selection for intrusion detection using random forest," *Journal of information security* vol. 7, no. 3, pp. 129–140, 2016.
- [6] Q. Ren, C. Hui, and H. Hai, "Research on machine learning framework based on random forest algorithm," *AIP conference proceedings* vol. 1820, no. 1, AIP Publishing, 2017.
- [7] S. Adelabu, M. Onisimo, and A. Elhadi, "Testing the reliability and stability of the internal accuracy assessment of random forest for classifying tree defoliation levels using different validation methods," *Geocarto International* vol. 30, no. 7, pp. 810–821, 2015.
- [8] N. Farnaaz, and M. A. Jabbar, "Random forest modeling for network intrusion detection system," *Procedia Computer Science* vol. 89, pp. 213–217, 2016.
- [9] A. Khraisat, et al. "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity* vol. 2, no. 1, pp. 1–22, 2019.
- [10] Y. Zhao, "Network intrusion detection system model based on data mining," *17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)* IEEE, 2016.